



CS
2nd
material

Network





Summary on Lecture 3

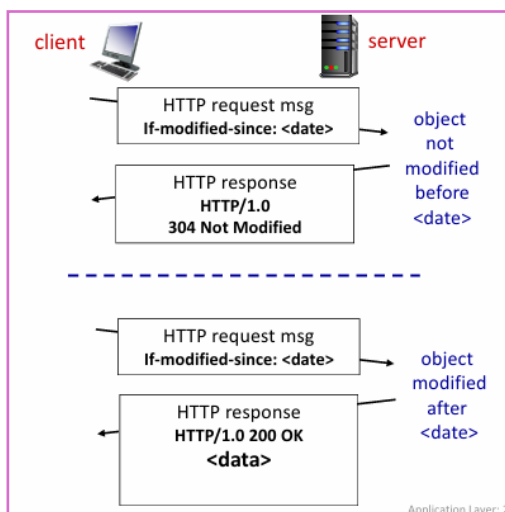
Why Web caching?

- reduce response time for client request
 - cache is closer to client
- reduce traffic on an institution's access link
- Internet is dense with caches
 - enables "poor" content providers to more effectively deliver content

○ تكلمه علي المحاضرة ال فانت

وقفنا اخر مرة عند ال Web Cache وهو بيشتغل عند ال Client و ال Server

- بيخلي الانترنت متبقاش مزحومه بكلام ملوش لازمه



- طيب ال Caching دا بيتم ازاي؟ السيناريو الأول:

○ ال Client (مثلاً متصفح الإنترنت) بيعت طلب HTTP للسيرفر، وبيستخدم ال If-Modified-Since عشان يسأل السيرفر لو كان فيه أي تحديث للملف أو البيانات اللي طالباها بعد تاريخ معين.

○ السيرفر بيشفوف لو البيانات اللي عنده ما اتعدلتش بعد التاريخ اللي ال Client حطه.

○ في الحالة دي، السيرفر بيرد على ال Client بـ HTTP/1.0 304 Not Modified، وده معناه إن مفيش تحديث حصل، وبيطلب من ال Client يستخدم البيانات اللي عنده من ال Cache (الذاكرة المؤقتة) بدل ما ينزلها تاني.

السيناريو الثاني:

○ ال Client بيعت طلب تاني برضه فيه If-Modified-Since بتاريخ معين.

○ لو السيرفر لقي إن فيه تحديثات حصلت للبيانات بعد التاريخ ده، بيرد على ال Client بـ HTTP/1.0 200 OK ومعها البيانات الجديدة.

○ ال Client في الحالة دي بياخد البيانات اللي نزلت ويحدث ال Cache عنده.

باختصار: الصورة بتوضح إن لو البيانات ما اتغيرتش من آخر مرة، السيرفر بيوفر على ال Client تحميلها تاني، لكن لو حصل تعديل، بيعتها له جديدة.

HTTP/2

Key goal: decreased delay in multi-object HTTP requests

HTTP/1.1: introduced **multiple, pipelined GETs** over single TCP connection

- server responds *in-order* (FCFS: first-come-first-served scheduling) to GET requests
- with FCFS, small object may have to wait for transmission (**head-of-line (HOL) blocking**) behind large object(s)
- loss recovery (retransmitting lost TCP segments) stalls object transmission

--> طيب عندنا حاجة اسمها HTTP/2 وهدفه الأساسي اللي هو تقليل التأخير في طلبات HTTP اللي فيها كذا حاجة بتتحمل في نفس الوقت

❖ HTTP/1.1 الإصدار القديم :

كان بيستخدم حاجة اسمها pipelining عشان بيعت أكثر من طلب (GET) في نفس الاتصال.

يعني بدل ما يفتح اتصال جديد لكل طلب، بيعتهم كلهم في نفس الاتصال.

- المشكلة كانت في إنه بيستخدم نظام (FCFS (First-Come-First-Served، وده يعني إنه لازم السيرفر يرد على الطلبات بنفس ترتيب وصولهم.

2. طيب ايه المشاكل ال بتواجهنا ف الطريقة دي :

- الـ: HOL Blocking (Head-of-Line Blocking) لو في طلب صغير وطلب كبير في نفس الاتصال، الطلب الصغير ممكن يضطر يستنى لحد ما الطلب الكبير يتبع الأول. وده معناه إن الطلبات الصغيرة بتتأخر لمجرد إنها في طابور انتظار الطلبات الأكبر.

- استرجاع البيانات الضائعة (Loss Recovery): لو حصل فقدان في أي جزء من البيانات أثناء الإرسال، بيحصل إعادة إرسال، وده بيعطل نقل بقية البيانات لحد ما المشكلة تتحل.

عشان نختصر كل الكلام دا HTTP/2: جه عشان يحل المشاكل دي ويقلل التأخير عن طريق تحسين طريقة الإرسال، بحيث الطلبات الصغيرة متستناش الكبيرة، ولو فيه جزء ضاع، التصليح يتم بدون ما يعطل نقل باقي البيانات

HTTP/2

Key goal: decreased delay in multi-object HTTP requests

HTTP/2: [RFC 7540, 2015] increased flexibility at *server* in sending objects to client:

- methods, status codes, most header fields unchanged from HTTP 1.1
- transmission order of requested objects based on client-specified object priority (not necessarily FCFS)
- push* unrequested objects to client
- divide objects into frames, schedule frames to mitigate HOL blocking

--> هنا هنكمل الشرح عن HTTP/2

ونوضح مميزاته الرئيسية مقارنة بـ HTTP/1.1 عشان تقلل التأخير في تحميل عدة ملفات في نفس الوقت.

1. سهولة أكثر في إرسال الملفات:

- HTTP/2 بيقدم مرونة أكبر للسيرفر في إرسال الملفات لل Client

- معظم ال (methods) و (status codes) و ال (header fields) اللي كانت في HTTP/1.1 لسه موجودة، لكن دلوقتي في مرونة أكبر في ترتيب وإدارة الإرسال.

2. ترتيب الإرسال حسب الأولوية:

- HTTP/2 يسمح للسيرفر يحدد ترتيب الإرسال بناءً على الأولويات التي يحددها Client، يعني مش شرط يمشي بنفس نظام الـ FCFS التي كان يفرض الرد علي حسب أول طلب يوصل.

3. إرسال ملفات بدون طلب مسبق:

- السيرفر يقدر يستخدم push ويبيع ملفات معينة للـ Client حتى لو الـ Client مطلبهاش. وده بيكون مفيد لو السيرفر عارف إن الـ Client هيجتاج الملفات دي بعد كده.

4. تجزئة الملفات لفريمات (Frames):

- الملفات الكبيرة بيتم تقسيمها لأجزاء صغيرة (فريمات)، وده بيخلي النظام يقدر يرتبهم عشان يقلل الـ (HOL Blocking) التي كان بيحصل لما الطلبات الصغيرة بتستنى الطلبات الكبيرة

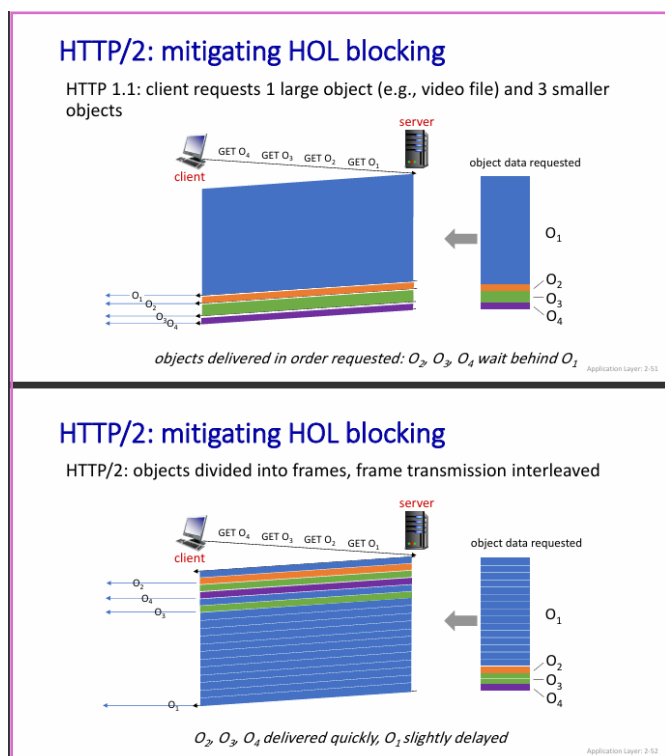
باختصار شديد HTTP/2: بيحاول يقلل التأخير عن طريق إدارة أولوية الطلبات، وإرسال الملفات المتوقعة بدون طلب مسبق، وتجزئة الملفات الكبيرة لأجزاء صغيرة عشان تكون عملية النقل أسرع وأكفاً.

■ هنعمل مقارنة سريعة بين الطريقتين: دول :

- الطريقة الأولى (HTTP/1.1): دي الطريقة القديمة، لما تطلب حاجات كتير من موقع واحد، زي مثلاً لما تفتح صفحة فيها صور وفيديو، الحاجات دي بتتنزل واحدة ورا الثانية. يعني لو في فيديو كبير، الفيديو ده هيقعد يتنزل الأول، والحاجات الثانية الصغيرة هتستنى لحد ما الفيديو يخلص عشان تبتي تنزل. ده بيخلي الصفحة تاخذ وقت طويل عشان تظهر بالكامل.

- الطريقة الثانية (HTTP/2): دي الطريقة الحديثة والأسرع، فيها بتتقسم البيانات التي بتطلبها لأجزاء صغيرة، والأجزاء دي بتتنزل في نفس الوقت، مش واحدة ورا

الثانية. يعني الفيديو الكبير بيتقسم لأجزاء صغيرة، والأجزاء دي بتتنزل جنب الحاجات الثانية الصغيرة. كده كل حاجة بتتنزل بسرعة، والصفحة بتظهر بشكل أسرع.



--> هنا بنقارن بين طريقتين للاتصال بالإنترنت: HTTP/2 و HTTP/3

HTTP/2 to HTTP/3

HTTP/2 over single TCP connection means:

- recovery from packet loss still stalls all object transmissions
 - as in HTTP 1.1, browsers have incentive to open multiple parallel TCP connections to reduce stalling, increase overall throughput
- no security over vanilla TCP connection
- **HTTP/3**: adds security, per object error- and congestion-control (more pipelining) over UDP
 - more on HTTP/3 in transport layer

HTTP/2: دي طريقة أحدث من الطريقة القديمة (HTTP/1.1) وبتخلي تحميل الصفحات أسرع وأكثر كفاءة. بس فيها مشكلة، لو حصل خلل في نقل البيانات، كل البيانات اللي بعدها بتقف لحد ما المشكلة تتصلح. ده زي ما يكون في طريق واحد بس للعربيات، ولو عريبه عطلت، كل العربيات اللي وراها هتضطر تقف.

HTTP/3: دي أحدث طريقة وهي تحسين على HTTP/2. بتحل مشكلة التوقف اللي في HTTP/2، يعني لو حصل خلل في نقل جزء من البيانات، الأجزاء التانية هتكمل بشكل طبيعي. ده زي ما يكون في طرق كثيرة للعربيات، فلو عريبه عطلت، العربيات التانية ممكن تستخدم طرق تانية وتكمل طريقها.

الفرق الكبير بين HTTP/2 و HTTP/3 هو:

* HTTP/2: بيستخدم طريقة واحدة لنقل البيانات (TCP)، ولو حصل خلل، كل البيانات بتتأثر.

* HTTP/3: بيستخدم طريقة أحدث وأكثر مرونة (UDP)، فلو حصل خلل في جزء من البيانات، الأجزاء التانية بتكمل بشكل طبيعي.

ببساطة، HTTP/3 أسرع وأكثر أماناً من HTTP/2، لأنه يسمح بنقل البيانات بشكل أكثر كفاءة ويقلل من تأثير الأخطاء.

➤ Email :-

- Three major components:

- User Agents
- Mail Servers
- Simple Mail Transfer Protocol: SMTP

- الفكرة من الايميل انا ك يوزر لازم استخدم برنامج بسميه ال User Agent زي ال اوتلوك ال User Agent دا لازم هيكلم حاجه اسمها ال Mail Server ال هو عليه الايميل ال Mail Server دا بيبقي جواه Mail Boxes بيحتفظ بالميلز بتاعة اليوزر و كمان بيحتوي علي Message Queue دي بيتحت

فيها المسدجز ال عايز يبعثها

ال Mail Servers دي كلها بتكلم بعض عن طريق ال SMTP

• ال SMTP بيشتغل علي ال TCP ودايما

بيشتغل علي Port 25 بيكون متقسم ل 3

مراحل :-

mail servers:

- **mailbox** contains incoming messages for user
- **message queue** of outgoing (to be sent) mail messages

SMTP protocol between mail servers to send email messages

- **client**: sending mail server
- **"server"**: receiving mail server

1- SMTP handshaking (greeting)

2- SMTP transfer of messages

3- SMTP closure

o SMTP requires message (header & body) to be in 7-bit ASCII

➤ Comparison Between HTTP and SMTP:

- HTTP: client pull
- SMTP: client push
- both have ASCII command/response interaction, status codes
- HTTP: each object encapsulated in its own response message
- SMTP: multiple objects sent in multipart message

→ Retrieving email: mail access protocols:-

- ايه ال بيحصل ف المرحلة دي؟
بكل بساطة ، تخيل نفسك بتبع رسالة لصاحبك، ده اللي بيحصل:
 - 1. بعث الرسالة :لما بتضغط على "إرسال"، بتروح الرسالة لمخزن مؤقت (زي صندوق بريد كبير) في سيرفر البريد بتاعك.
 - 2. سيرفر البريد :ده زي مكتب البريد، بيأخذ الرسالة وبيوديهها لصندوق البريد بتاع صاحبك في سيرفر ثاني.
 - 3. استقبال الرسالة :لما صاحبك بيفتح عشان يشوف رسائله، بيستخدم برنامج خاص (زي جيميل أو أوتلوك) عشان يتصل بسيرفر البريد ويجيب الرسالة.
 - الجزء المهم:
 - SMTP : ده البروتوكول اللي بيستخدم في إرسال الرسائل بين السيرفرات.
 - IMAP او POP : دول بروتوكولات بتستخدم عشان تجيب الرسائل من السيرفر على جهازك.
 - HTTP : ده البروتوكول اللي بيستخدم في المواقع الإلكترونية، وبستخدموه بعض برامج البريد عشان تعرض الرسائل في المتصفح (زي جيميل).
 - > تلخيصا :
 - SMTP : زي ساعي البريد اللي بيوصل الرسائل من باب لباب.
 - IMAP و POP : زي العربي اللي بتجيب لك كل رسائلك دفعة واحدة.
 - HTTP : زي صندوق البريد اللي بتفتحه عشان تشوف رسائلك.
- ملحوظة** :في فرق بين IMAP و POP ، بس الفكرة الأساسية هي إنهم بيستخدموا عشان تجيب الرسائل من السيرفر.

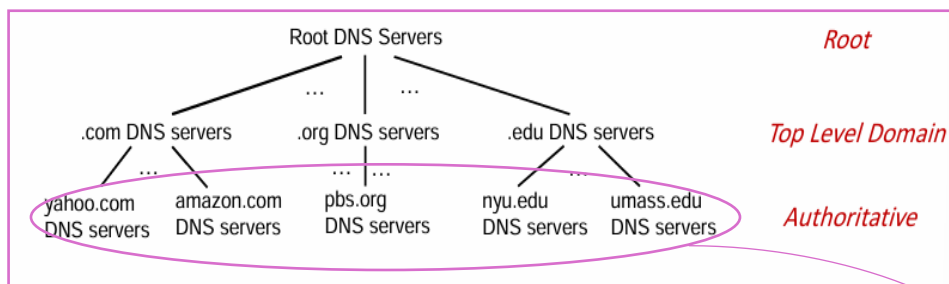
→ Domain Name System (DNS):-

Domain Name System (DNS):

- *distributed database* implemented in hierarchy of many *name servers*
- *application-layer protocol*: hosts, DNS servers communicate to *resolve* names (address/name translation)
 - *note*: core Internet function, implemented as *application-layer protocol*
 - complexity at network's "edge"

- بكل بساطة ، ده زي دليل تليفون الإنترنت. بدل ما تحفظ أرقام تليفونات كل الناس، بتفتكر أسمائهم عشان تتصل بيهم، صح؟ نفس الفكرة هنا، بس بدل أرقام تليفونات، بنستخدم عناوين رقمية (IP Addresses) عشان الأجهزة تتواصل مع بعض على الإنترنت.

- ال Root DNS Server بيص علي ال TLD (Top Level Domain) وهو متسجل عنده كل TLD مين السيرفر ال بتخدمه يعني



لو مبعوتلي domain اخره .com. بيقى اروح للسيرفرز دي وهكذا

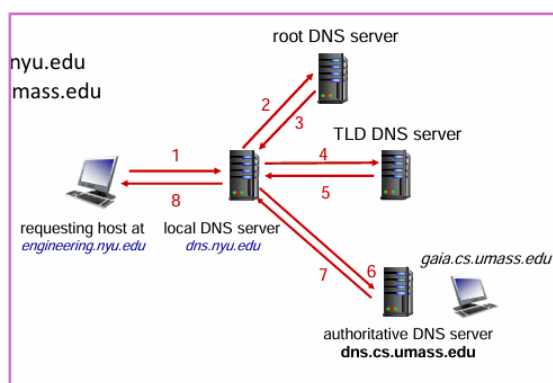
- عندنا 13 Root Server دول ال بيخدموا العالم كله

→ Authoritative DNS Servers

Local DNS name servers

- when host makes DNS query, it is sent to its *local* DNS server
 - Local DNS server returns reply, answering:
 - from its local cache of recent name-to-address translation pairs (possibly out of date!)
 - forwarding request into DNS hierarchy for resolution
 - each ISP has local DNS name server; to find yours:
 - MacOS: `% scutil --dns`
 - Windows: `>ipconfig /all`
- local DNS server doesn't strictly belong to hierarchy

تخيل إنك عايز تدخل على موقع جوجل، بدل ما جهازك يروح يسأل كل السيرفرات في العالم عشان يعرف عنوان جوجل، بيروح أولاً يسأل ال Local DNS Server. لو السيرفر ده عارف العنوان، هيقوله على طول، ولو مش عارفه، هيروح يسأل السيرفرات الأكبر عشان يجيب المعلومة ويخزنها عنده، عشان المرة الجاية يعرف يرد عليك بسرعة.



- لنفترض اني عايز ادخل علي

الويبسايت دا engineering.nyu.edu

فهبت الريبكويست لل DNS Server ف هيدور علي ال TLD ملقهاش هيروح بيعت لل Root Server يقوله انا عايز ادور علي كذا ف الروت يرد عليه بالنتيجة ف السيرفر يروح يكلم ال TLD ويرد عليه وهكذا بقي

DNS: distributed database storing resource records (RR)

RR format: (name, value, type, ttl)

- Type = A
 - Name is Hostname
 - Value is IP Address
- Type = CNAME
 - Name is alias name for some "canonical" (the real) name
 - Value is canonical name
- Type = NS
 - Name is domain (e.g., foo.com)
 - Value is hostname of authoritative name server for this domain
- Type = MX
 - Value is name of SMTP mail server associated with Name

DNS security

DDoS attacks

- bombard root servers with traffic
 - not successful to date
 - traffic filtering
 - local DNS servers cache IPs of TLD servers, allowing root server bypass
- bombard TLD servers
 - potentially more dangerous

Spoofing attacks

- intercept DNS queries, returning bogus replies
 - DNS cache poisoning
 - RFC 4033: DNSSEC authentication services

- DDOS : دي زي لو حد قرر يسد طريقك عشان متوصلش لمكانك. هنا، الهاكرز بيحاولوا يسدوا السيرفرات الرئيسية لل Local DNS بكمية كبيرة من الطلبات، عشان يمنعوا أي حد من الوصول للإنترنت.
- Spoofing : دي زي لو حد انتحل شخصيتك عشان يخدع حد ثاني. هنا، الهاكرز

بيحاولوا يخدعوا الأجهزة ويوجهوها لمواقع وهمية بدل المواقع الحقيقية اللي بتدور عليها. ده ممكن يحصل عن طريق عدة طرق، زي (DNS cache poisoning)

→ Peer-To-Peer (P2P) architecture:-

- مش بيبقي عندي Server عليه برنامج لانه مش 2 Programs هو Single Program بيبقي متحمل علي ال end systems ويحصل communication دايركت بين البرامج دي

File distribution time: client-server

- **server transmission:** must sequentially send (upload) N file copies:
 - time to send one copy: F/u_s
 - time to send N copies: NF/u_s



- **client:** each client must download file copy

- d_{min} = min client download rate
- min client download time: F/d_{min}

$$D_{c-s} \geq \max\{NF/u_s, F/d_{min}\}$$

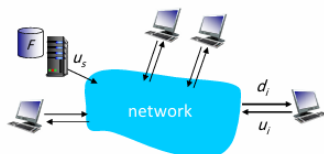
increases linearly in N

ابطأ واحد فيهم

- علشان نقدر نعمل داونلود من السيرفر الحل الوحيد ان كل واحد يدخل يعمل داونلود للفايل علشان ياخذ ال Full Speed بتاعته وبعد ما يخلص هياخذ ال بعده وهكذا
- طيب لو انا الوقتي بعمل داونلود للفايل من السيرفر وبستخدم ال Full Speed بتاعة السيرفر دا معناه ان ال Download Speed بتاعتي اكبر من او يساوي ال Upload Speed بتاعة السيرفر

File distribution time: P2P

- **server transmission:** must upload at least one copy:
 - time to send one copy: F/u_s
- **client:** each client must download file copy
 - min client download time: F/d_{min}
- **clients:** as aggregate must download NF bits
 - max upload rate (limiting max download rate) is $u_s + \sum u_i$



$$D_{P2P} \geq \max\{F/u_s, F/d_{min}, NF/(u_s + \sum u_i)\}$$

increases linearly in N ...
... but so does this, as each peer brings service capacity

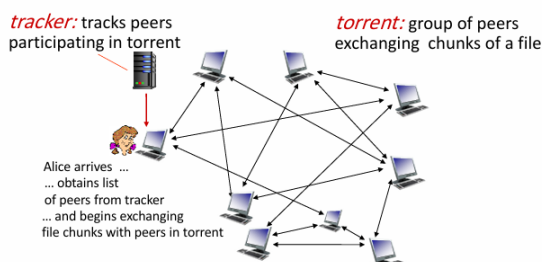
Application Layer

واحد يعمل داونلود لجزء لحد ما نقسم الفايل عليهم كلهم

- هنا انا هخلي الفايل موجود علي السيرفر واخلي كل ال Clients يعملوا داونلود في نفس الوقت بس مش كلهم هيعملوا داونلود للفايل نفسه كل واحد هينزل جزء من الفايل مختلف كاني هقسم الفايل دا ل Chunks وكل واحد هياخذ Chunk وكل

P2P file distribution: BitTorrent

- file divided into 256Kb chunks
- peers in torrent send/receive file chunks



- هنا ايه ال بيحصل؟

- 1- اول حاجه هنقسم الملف لأجزاء صغيرة بنسميها (chunks).
- 2- ثاني حاجه كل شخص بيشترك جزء من الملف مع غيره والعكس صحيح.
- 3- ايه ال (Tracker) دا؟ دا زي قائمة حضور، بيعرف مين موجود في المجموعة اللي بتنزل الملف وببوجه كل واحد لواحد ثاني علشان ياخذ منه الأجزاء اللي ناقصاه.
- 4- (Torrent) : دي مجموعة من الناس اللي بتنزل نفس الملف، وكل واحد فيهم بيشترك الأجزاء اللي عنده مع الباقي.

--> ال BitTorrent Protocol مبني علي حاجه اسمها ال tit-for-tat بمعني اني النهارده داخل علي جهاز بعمل داوولود منه ل chunks معينه اول حاجه الجهاز بيعملها انه بيدور علي الحاجه ال عايز يعملها داوولود لو لقاها عندي بيتدي يفتحي ال Speed بتاعة الداوولود (زي خد وهات كدا) طيب لو بعمل داوولود من عند شخص وملقاش حاجه ياخذها مني هيعمل ايه؟ هيديني الفايل بسرعه محدده وقليله.

❖ طول بالك معانا يغالي لسه حبه

--> ف ال Video Stream بيقابلنا شوية مشاكل كدا ايه هي؟

- اول حاجه الفيديوها بتستهلك جزء كبير جدا من الانترنت تخيل مثلاً كمية الناس اللي بتشوف أفلام ومسلسلات على YouTube وNetflix كثير جدا طبعاً

- ف ايه بقي الحل؟

اننا نستخدم ال CDNs طب ايه ال CDNs دي؟ خليني اوضحها لك بمثال بسيط

تخيل إنك عايز تشتري كتاب من مكتبة بعيدة، بدل ما تروح للمكتبة دي، ممكن تروح لمكتبة أقرب ليك وتطلب الكتاب، وهما هيجيبوه لك من المكتبة الرئيسية. ال CDNs بقي بتعمل نفس الشيء بالضبط، بس مع الفيديوها وهي السبب عموماً ف اننا بنعرف نشوف الفيديوها أونلاين بشكل سريع وسلس

→Multimedia: video

- اول حاجه الفيديو عندنا مش صورة واحده هو عبارة عن مجموعه من الصور بتتغير بسرعه علشان تعمل حركه والصورة الواحده دي بتتكون من نقط صغيره بنسبها
Pixels كل بكسل بتتمثل ف bit

■ coding: use redundancy **within** and **between** images to decrease # bits used to encode image

- spatial (within image)
- temporal (from one image to next)

- بيبحت عن أجزاء متشابهة في نفس الصورة ويقلل من حجم البيانات

- يقارن بين صورتين متتاليتين ويخزن بس الأجزاء اللي اتغيرت

➤ Streaming multimedia: DASH(Dynamic, Adaptive Streaming over HTTP)

- دي طريقه بتخلي مشاهده الفيديوها اسهل وافضل بكثير طب بتشتغل ازاى؟

- 1- اول حاجه بيقسموا الفيديو إلى أجزاء صغيرة
- 2- ثاني حاجه بيوزعوا الأجزاء دي على سيرفرات مختلفة في كل العالم علشان يكون الوصول أسرع .
- 3- Manifest File : الجهاز اللي بتشوف عليه الفيديو بيختار باستمرار أفضل جودة للفيديو بناءً على سرعة الإنترنت المتاحة عنده.

➤ Socket Programming:-

- زي ماحنا عارفين ان ال Socket هي المكتبة ال بستخدمها علشان ابعت واستقبل رسائل

Two socket types for two transport services:

- **UDP:** unreliable datagram
- **TCP:** reliable, byte stream-oriented

Socket programming with UDP

UDP: no "connection" between client and server:

- no handshaking before sending data
- sender explicitly attaches IP destination address and port # to each packet
- receiver extracts sender IP address and port# from received packet

UDP: transmitted data may be lost or received out-of-order

Application viewpoint:

- UDP provides *unreliable* transfer of groups of bytes ("datagrams") between client and server processes

- هنا ال UDP زي البريد العادي مش محتاج عنوان ثابت يعني مش لازم تعرف عنوان المستقبل بالظبط علشان تبعت له رسالة، بس لازم تحدد عنوانه في كل رسالة بتبعتها ، ممكن الرسالة تضيع في الطريق أو توصل متأخرة أو حتى توصل أجزاء منها قبل أجزاء تانية

يعني تخيل أنك بتلعب لعبة اونلاين

لو استخدمت بروتوكول TCP ، أي تأخير بسيط في الاتصال ممكن يؤدي إلى بطء اللعبة أو تقطيعها. لكن باستخدام UDP ، حتى لو ضاعت بعض البيانات، اللعبة هتستمر في العمل بشكل سلس.

Socket programming with TCP

Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

Client contacts server by:

- Creating TCP socket, specifying IP address, port number of server process
- *when client creates socket:* client TCP establishes connection to server TCP

- when contacted by client, *server TCP creates new socket* for server process to communicate with that particular client
 - allows server to talk with multiple clients
 - client source port # and IP address used to distinguish clients (more in Chap 3)

Application viewpoint

TCP provides reliable, in-order byte-stream transfer ("pipe") between client and server processes

- هنا ال Client لازم يتواصل مع (السيرفر):
- في البداية، لازم عملية السيرفر تكون شغالة علشان تستقبل اتصالات ال Clients (يعني تكون فاتحة باب أو منفذ).
- السيرفر لازم يكون عمل Socket (زي باب للاتصال) علشان يقدر يستقبل اتصالات ال Clients اللي هيجوا يتواصلوا.

طيب ال Client بيتواصل مع السيرفر إزاي؟

- ال Client بيعمل Socket (زي وصلة أو باب) ويحدد فيها عنوان IP بتاع السيرفر ورقم ال (Port) اللي السيرفر بيستمع عليه.
- لما ال Client يخلق ال Socket دي، ببدا عملية الاتصال بالسيرفر من خلال بروتوكول TCP.
- لما ال Client يتصل، إيه اللي بيحصل؟

- السيرفر يخلق Socket جديد مخصوص لكل Client عشان يقدر يتكلم معاه بشكل منفصل. كده يقدر يتعامل مع أكثر من Client في نفس الوقت.
- السيرفر بيستخدم رقم ال Port وعنوان ال IP لل Client عشان يقدر يميز بين ال Clients المختلفين.



لا تنسو الدعاء لآخواننا في
فلسطين و غزة و العراق و
لبنان و سوريا و السودان.
و تذكر دائما المقاطعة ولا
تتخلي عن مبادئك و انت بتجاهد
معاهم بمقاطعتك.